

Informe de Laboratorio 02

Tema: Formularios Web, Validaciones Nativas y CSS3

Nota

Estudiante	Escuela	Asignatura
Edu Breyner Mio Alviz emioa@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Introducción al Desarrollo Web Semestre: II Código: 20251174

Laboratorio	Tema	Duración
02	Formularios Web, Validaciones Nativas y CSS3	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - I	Del 07 Septiembre 2026	Al 13 Septiembre 2026

1. Tarea

- Crear un formulario web para el registro de estudiantes utilizando HTML.
- Implementar validaciones nativas mediante atributos HTML5.
- Utilizar diferentes tipos de controles de formulario como texto, correo, contraseña, fecha, selección, radio, checkbox y textarea.
- Aplicar estilos mediante un archivo CSS externo.
- Utilizar el modelo de caja de CSS mediante la propiedad `box-sizing: border-box`.
- Implementar estados CSS como `:hover`, `:focus`, `:active` y `:disabled`.
- Comprobar el funcionamiento del formulario mediante el navegador y sus herramientas de desarrollo.

2. Equipos, materiales y temas utilizados

- Computadora personal.
- Sistema operativo Windows 11.
- Visual Studio Code.
- Navegador web Opera y google

- GitHub.
- HTML5.
- CSS3.
- GitHub Pages.
- Herramientas de desarrollador del navegador.
- Formularios HTML.
- Validaciones nativas HTML5.
- Modelo de caja CSS.

3. URL de Repositorio Github

- URL del repositorio GitHub:
- `https://github.com/emioa-bit/idweb`
- URL del Laboratorio 02:
- `https://github.com/emioa-bit/idweb/tree/main/Lab02`
- URL de GitHub Pages:
- `https://emioa-bit.github.io/idweb/Lab02/`

4. Actividades con el repositorio GitHub

4.1. Creación de la estructura del laboratorio

Se creó la carpeta correspondiente al Laboratorio 02 dentro del repositorio GitHub.
La estructura utilizada fue la siguiente:

```
1 IDWeb/  
2 |--- Lab02/  
3 |   |--- index.html  
4 |   |--- css/  
5 |       |--- styles.css
```

4.2. Creación del formulario HTML

Se creó el archivo `index.html`, donde se implementó el formulario para el registro de estudiantes.
El formulario fue organizado mediante los elementos `fieldset` y `legend`.

Listing 1: Estructura del formulario

```
1 <form action="#" method="get">  
2  
3   <fieldset>  
4     <legend>Informacin Personal</legend>  
5  
6     <label for="codigo">
```

```
7      Codigo de estudiante:
8      </label>
9
10     <input type="text"
11           id="codigo"
12           name="codigo"
13           pattern="[0-9]{8}"
14           required
15           title="El cdigo debe tener exactamente 8 dgitos">
16
17     </fieldset>
18
19 </form>
```

4.3. Validación del código de estudiante

Se utilizó el atributo `pattern` para permitir únicamente un código formado por exactamente ocho números.

Listing 2: Validación del código

```
1 <input type="text"
2       id="codigo"
3       name="codigo"
4       pattern="[0-9]{8}"
5       required
6       title="El cdigo debe tener exactamente 8 dgitos">
```

El patrón utilizado es:

```
1 pattern="[0-9]{8}"
```

Esto significa que el código debe contener exactamente ocho caracteres numéricos.
Por ejemplo:

- **12345678**: válido.
- **1234567**: inválido.
- **123456789**: inválido.
- **ABC12345**: inválido.

4.4. Validación de contraseña

Para la contraseña se utilizó una expresión regular que exige como mínimo ocho caracteres, una letra mayúscula y un número.

Listing 3: Validación de contraseña

```
1 <input type="password"
2       id="password"
3       name="password"
4       pattern="(?=.*[A-Z])(?=.*[0-9]).{8,}"
5       required
6       title="Mnimo 8 caracteres, una mayscula y un nmero">
```

Por ejemplo:

- **Edu12345**: válido.
- **edu12345**: inválido porque no contiene una mayúscula.
- **EduBreyner**: inválido porque no contiene un número.

4.5. Validación de fecha

Se utilizaron los atributos `min` y `max` para establecer un rango permitido para la fecha de nacimiento.

Listing 4: Validación de fecha

```
1 <input type="date"
2     id="fecha"
3     name="fecha"
4     min="1990-01-01"
5     max="2008-12-31"
6     required>
```

De esta manera, el navegador impide seleccionar fechas que estén fuera del rango establecido.

4.6. Información de la cuenta

Se agregaron los campos correspondientes a correo electrónico, contraseña, aceptación de términos y comentarios.

Listing 5: Información de cuenta

```
1 <fieldset>
2
3     <legend>Información de la Cuenta</legend>
4
5     <label for="email">
6         Correo electrónico:
7     </label>
8
9     <input type="email"
10         id="email"
11         name="email"
12         required>
13
14     <label for="password">
15         Contraseña:
16     </label>
17
18     <input type="password"
19         id="password"
20         name="password"
21         required
22         minlength="8">
23
24     <label>
25         <input type="checkbox"
26             name="terminos"
27             required>
```

```
28
29     Acepto los trminos y condiciones
30 </label>
31
32 <label for="comentarios">
33     Comentarios:
34 </label>
35
36 <textarea id="comentarios"
37         name="comentarios"
38         rows="5">
39 </textarea>
40
41 </fieldset>
```

5. Estilizado mediante CSS

Se creó el archivo externo:

```
1 css/styles.css
```

El archivo fue conectado desde el documento HTML mediante:

Listing 6: Enlace del archivo CSS

```
1 <link rel="stylesheet" href="css/styles.css">
```

5.1. Box Model

Se utilizó:

Listing 7: Uso de border-box

```
1 * {
2     box-sizing: border-box;
3 }
```

La propiedad `border-box` permite que el ancho y alto definidos incluyan el contenido, el padding y el borde.

5.2. Diseño de la tarjeta

El formulario fue presentado dentro de una tarjeta centrada, con espaciado interno, bordes redondeados y sombra.

Listing 8: Diseño del formulario

```
1 form {
2     max-width: 600px;
3     margin: 0 auto;
4     background-color: white;
5     padding: 25px;
6     border-radius: 10px;
7     box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
```

8 }

5.3. Estados de los elementos

Se utilizaron diferentes pseudoclases de CSS para mejorar la interacción del usuario.

Listing 9: Estados CSS

```
1 input:hover,
2 select:hover,
3 textarea:hover {
4     border-color: #666;
5 }
6
7 input:focus,
8 select:focus,
9 textarea:focus {
10     outline: none;
11     border-color: #3182ce;
12 }
13
14 button:hover {
15     opacity: 0.8;
16 }
17
18 button:active {
19     transform: scale(0.90);
20 }
21
22 input:disabled {
23     background-color: #ddd;
24     cursor: not-allowed;
25 }
```

El estado `:active` permite observar un cambio visual mientras el botón se encuentra presionado.

5.4. Estructura final del Laboratorio 02

La estructura final del laboratorio quedó organizada de la siguiente manera:

```
1 IDWeb/
2 |--- Lab02/
3 |   |--- index.html
4 |   |--- css/
5 |       |--- styles.css
```

6. Ejecución y pruebas

Se publicó el formulario mediante GitHub Pages y se realizaron diferentes pruebas de validación.

6.1. Prueba del código de estudiante

Se introdujo un código de siete dígitos:

1 1234567

El navegador mostró un mensaje indicando que el valor no cumplía con el formato requerido.
Posteriormente se probó:

1 12345678

El valor fue aceptado debido a que contiene exactamente ocho dígitos.

6.2. Prueba de contraseña

Se probó una contraseña sin letra mayúscula:

1 edu12345

El navegador indicó que el valor no cumplía con el patrón establecido.
Luego se utilizó:

1 Edu12345

La contraseña fue aceptada.

6.3. Prueba de términos y condiciones

Se intentó enviar el formulario sin marcar la casilla:

1 Acepto los trminos y condiciones

El navegador solicitó marcar el checkbox debido al atributo **required**.

7. Evidencias

7.1. Evidencia del formulario

En esta sección se debe incluir una captura de pantalla donde se observe el formulario funcionando correctamente.

Figura 1: Formulario de registro de estudiantes.

7.2. Evidencia de las validaciones

Se debe incluir una captura donde se observe una validación HTML5 del formulario.

Figura 2: Validación nativa del formulario.

7.3. Evidencia de la pestaña Network

Se debe incluir una captura de la pestaña **Network** del navegador donde se pueda observar una solicitud HTTP y su código de estado.

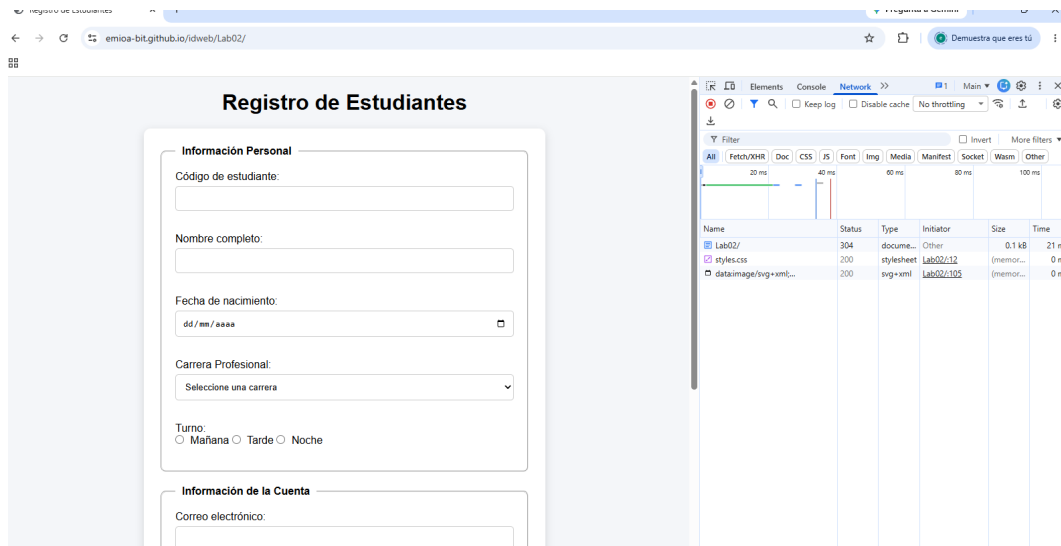


Figura 3: Solicitud HTTP observada en la pestaña Network.

8. Preguntas

8.1. ¿Cuál es la diferencia entre content-box y border-box?

content-box considera el ancho y alto únicamente para el contenido del elemento. El padding y el borde se agregan al tamaño total.

En cambio, **border-box** incluye dentro del ancho y alto definidos el contenido, el padding y el borde.

Por ello, **border-box** facilita el control del tamaño de los elementos dentro de una página web.

8.2. ¿Cómo funciona la especificidad en CSS?

La especificidad determina qué regla CSS tiene mayor prioridad cuando existen varias reglas que afectan al mismo elemento.

De manera general, la prioridad puede expresarse como:

1. Estilos inline.
2. Selectores por ID.
3. Selectores por clase.
4. Selectores por etiqueta.

Por ejemplo, un selector por ID tiene mayor especificidad que un selector por clase.

8.3. ¿Cuál es la diferencia entre display: none y visibility: hidden?

display: none elimina visualmente el elemento y también hace que deje de ocupar espacio dentro del diseño de la página.

En cambio:

```
1 visibility: hidden;
```

oculta visualmente el elemento, pero mantiene el espacio que ocupaba originalmente.
Por lo tanto:

- `display: none`: no se muestra y no ocupa espacio.
- `visibility: hidden`: no se muestra, pero mantiene su espacio.

9. Conclusiones

- Se implementó un formulario web utilizando diferentes controles de HTML5.
- Se utilizaron validaciones nativas para controlar los datos introducidos por el usuario.
- Se aplicó CSS externo para mejorar la presentación del formulario.
- Se utilizó el modelo de caja mediante `box-sizing: border-box`.
- Se implementaron pseudoclases para mejorar la interacción del usuario.
- Se comprobó el funcionamiento del formulario mediante diferentes pruebas en el navegador.
- Se utilizó GitHub como medio para almacenar y publicar el trabajo realizado.

10. Rúbricas

10.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio, buena presentación y fácil de leer.

10.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en las celdas correspondientes de la columna Checklist si cumplió con el ítem correspondiente.
- El alumno debe autocalificarse en la columna Estudiante.

Tabla 2: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 3: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas.	4	X	3	
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	2	
4. Ejecución	Se incluyen ejecuciones y pruebas del código fuente explicadas gradualmente.	2	X	2	
5. Pregunta	Se responde con completitud a las preguntas formuladas en la tarea.	2	X	1	
6. Fechas	Las fechas de modificación del código fuente están dentro de los plazos de fecha de entrega establecidos.	2	X	2	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	1	
8. Madurez	El informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable.	4	X	2	
Total		20		20	

11. Referencias

- <https://developer.mozilla.org/es/docs/Web/HTML>
- <https://developer.mozilla.org/es/docs/Web/CSS>
- <https://www.w3schools.com/html/>
- <https://www.w3schools.com/css/>